ELI — Adaptive Inference Governor

ELI v2.1.92

August 2026

Contents

Adaptive Inference Governor — Plan

Status: Proposal (no code changed by this document) **Date:** 2026-06-17 **Scope:** Make ELI's reasoning/inference *policy* (think vs no-think, token budget, pass count, recovery) a pure function of the runtime environment + the task, so a single redistributed binary behaves correctly on any user's machine, with any model, with zero per-machine tuning. **Provenance:** Derived from the 2026-06-17 session log (`EXPLAIN_*`, `SELF_IMPROVE` empty-`<think>` cascades) and a full read of the existing adaptation substrate. Every factual statement below is cited to a file/line at commit `a641471`. Where the session log is the evidence, it is quoted.

> Accuracy note / corrections to earlier verbal claims made in chat, now fixed here: 1. `persona_updater` does **not** hold the inference lock — it is deterministic (DB + regex). Its cost is redundant CPU/DB churn, not LLM-lock contention. 2. The ctx table **does** match `qwen3` (→ 32768). The real defect is different: it *undershoots* the model's true trained context (`n_ctx_train=262144` in the log). 3. `record_speed` is called on the **non-stream** path only, not "every generation." 4. The CoT fallback's empty-`<think>` is **model-emergent**, not injected by ELI.

---

0. The one-sentence problem

ELI adapts its inference to **memory** (VRAM, context length) and **model size**, but **not to compute speed (tokens/sec) or to the loaded model's intrinsic capabilities**, and the think/no-think decision is split between a global env toggle and a single hard-coded call site — so on a slow or unfamiliar machine the same prompt that works on the dev box produces 5–9 minute turns and empty answers.

The session log shows this concretely: `EXPLAIN_COGNITION_RUNTIME` burned a 273 s empty compact-synthesis pass, then **two** 539 s empty standard-synthesis passes, before a working two-stage CoT fallback (470 s + 502 s) — ~38 minutes for one question, most of it discarded.

1. What ELI already does dynamically (verified — do NOT rebuild)

The architecture for dynamism is largely present and correct. The governor must *extend* these, not replace them.

Capability	Where (verified)	What it adapts to
Model-agnostic resolution (no baked model)	`gguf_inference.get_model_path`; blueprint `inference_and_hardware.md`	settings/env
Family-aware chat templating	`gguf_inference._is_chatml_model` / `_is_llama3_model` / `_is_mistral_model` (≈195–230)	model **filename**
Free-VRAM-aware load sizing	`core/hardware_profile.py` (`HardwareProfile`, `_layers_for_size`)	free VRAM
Compute-graph reserve (model-agnostic)	`hardware_profile.py:69` → `256 + (n_ctx/1024)*24 + batch*1.5` MB	ctx, batch
KV-cache cost	`hardware_profile.py:46` `_kv_cache_mb`	ctx, layers
Graceful GPU-layer fallback (attempt list)	boot optimizer; see `arti-facts/runtime_snapshot.json` `adaptive_load_report.attempts`	load success
Context sizing per model	`core/startup_hardware_optimizer.py:164` `train_ctx_for_model`	model **filename**
Per-mode token budget	`cognition/reasoning_modes.py:312-352`	**n_ctx**, prompt pressure, query complexity, mode
Size tier → budget scale	`core/model_tier.py` `detect_tier`/`tier_scale` (by file GB)	model size
Live decode-speed EMA	`model_tier.py:90` `record_speed`, `:104` `measured_tok_s`	**measured tok/s**
Speed-capped pass COUNT	`model_tier.py:120` `speed_passes`; engine `6241` (ToT k/depth), `6520` (self-consistency n)	measured tok/s
No-think on utility calls	`gguf_inference._no_think_prefill`	structured JSON, small budget (<1024)
Redistribution-aware settings	`core/runtime_settings.py` (`_portable_settings_for_storage`, `_heal_model_paths`)	machine moves

Key existing fact #1 — a live throughput EMA already exists. `gguf_inference.py:1158–1164` records decode speed from the model's own `completion_tokens / elapsed` after every **non-stream** generation:

```
_gen = int((response.get("usage") or {}).get("completion_tokens") or 0) or max(1, len(_raw_text)//4)
if _elapsed > 0.05 and _gen >= 4:
    from eli.core.model_tier import record_speed as _rec_speed
    _rec_speed(_gen / _elapsed)
```

Key existing fact #2 — that EMA is consumed in exactly ONE place. `grep measured_tok_s` → only `model_tier.py:126`, inside `speed_passes()`. Nothing in the think decision, the token budget, or mode selection reads it. `model_tier.py` says so in its own docstring (lines 83–86): *"speed_passes() … only reduces how many full generations a mode runs. It NEVER caps output length."*

2. The gaps (each verified, with code + log evidence)

Gap A — Thinking-detection is a filename allowlist, not intrinsic

`gguf_inference.py:219 _is_thinking_model`:

```
return any(k in name for k in ("qwen3", "deepseek-r1", "r1-distill", "qwq", "-r1-"))
```

Chat **format** is detected from the model family, but whether the model emits a `<think>` block is matched against a hard-coded substring list. A redistributed ELI handed a reasoning model whose filename isn't on the list (a future family; a user-renamed `MyModel.gguf`; a re-quant) will not know it thinks → the no-think prefill never fires → the exact empty-`<think>` loop, on the user's machine, with no recourse and no toggle they'd know to flip. This is the same brittle pattern as the filename-keyed ctx table (Gap E).

Gap B — The think decision and per-call token budget ignore measured throughput

This is the root cause of the log's wasted ~38 minutes.

- The budget (`reasoning_modes.py:312–352`) is a function of `n_ctx`, prompt pressure, complexity, and mode. **No tok/s term.** Verified: the only inputs are `n_ctx`, char counts, and `prof`.
- The think decision (`_no_think_prefill`) keys on `structured`, `max_tokens < 1024`, the `force_no_think()` scope, and the `ELI_MODEL_THINK` env. **No tok/s term.**

Consequence on a slow box: a large-budget (≥1024) non-structured call keeps thinking, and the budget can be 1800–4096 tokens. At the log's effective speed that is 8–35 minutes per call. Derived speed estimate from the log (legitimate arithmetic, labelled as an estimate): the working CoT final answer was **4071 chars (~1018 tokens) in 502 s ≈ ~2 tok/s**. At 2 tok/s a 1800-token think budget is ~15 min before the model even reaches an answer — and if it doesn't close `</think>` first, the output is discarded (Gap D).

Gap C — No wall-time governor / projection

Nothing computes "at the live EMA this call ≈ N seconds" and adapts (clamp tokens, drop think, downgrade mode) to a latency target. `speed_passes()` reduces the *number* of passes but not the *cost of each pass*. The EMA needed to do the projection already exists (`measured_tok_s`); it is simply never used for this.

Gap D — Empty output is not self-correcting in a model-agnostic way

`gguf_inference._strip_think_text` (≈345–358) drops a never-closed `<think>` outright, and `_clean_*` returns the raw fallback, which is then empty. Log:

```
[GGUF][RAW_TEXT] '<think>\nHere's a thinking process:\n...'   (budget exhausted inside think)
[GGUF][CLEAN] Empty after cleaning, returning raw fallback
[COGNITIVE] Compact synthesis returned empty ... falling back to standard synthesis
[GGUF][TIMING] nonstream_call_total=539.994s  → empty (twice)
```

An unterminated-think outcome is detectable (`<think>` present, no `</think>`, hit max_tokens) but is not used to recover. The system pays full price, returns empty, then pays full price again. There is no "force-close + short continuation" or "immediate no-think retry" path keyed on this signal.

Gap E — Context sizing undershoots the model's real trained context

`train_ctx_for_model` (`startup_hardware_optimizer.py:186–197`) returns **32768** for any `qwen3`. The loaded model's actual trained context is larger — the log states it:

```
llama_context: n_ctx_seq (20480) < n_ctx_train (262144) -- the full capacity ... will not be utilized
```

So the filename table gave 32768 while the model supports 262144 (8× more). The model's own GGUF metadata (`n_ctx_train`) is authoritative and available at load; the filename table is a guess that can be

wrong both ways (over-asks → llama clamps; under-asks → wasted capacity). Same intrinsic-vs-filename principle as Gap A.

Gap F — `force_no_think` is a point-patch (wrong shape for redistribution)

`grep force_no_think` → defined in `gguf_inference.py` (≈255), called from exactly one site: `engine.py:13070` (the compact grounded-synthesis wrap committed in `a641471`). It fixes the `EXPLAIN_*` turns, but: - The **standard-synthesis fallback** `_synthesize_answer` (`engine.py:13118`, `max_tokens=1800` at ≈13165 for non-quick) still thinks → still able to empty. - The **coding-agent fix-generation** invoked by `SELF_IMPROVE/SELF_ANALYZE` (`runtime/self_improvement.py` → `eli/coding`) emits the same empties in the log (`max_tokens=1536/4000`, `<think>` → empty, 231 s / 445 s). Note `self_improvement`'s own `broker.infer(..., max_tokens=700)` (`self_improvement.py:641`) is small → already no-think; the empties are the larger coding-agent generations.

Hard-coding the decision per-call-site cannot scale to every path, and on a *fast* box the correct answer is the opposite (keep thinking). The decision must be derived, not pinned.

Gap G — Redundant proactive churn (secondary; corrected claim)

The log shows `persona_updater: overlay updated` + `kg sync complete — 50 entities, 49 relations` + `user profile updated` repeating dozens of times per turn. **Verified correction:** `persona_updater.py` is deterministic — `_populate_kg_from_user_patterns` (:543), `update_persona_overlay` (:349) do DB reads + regex; the only LLM reference (:268) *reads* existing summaries. So this is **not** inference-lock contention (my earlier verbal claim was wrong) — it is redundant recomputation (same 50/49 KG rebuilt every tick) burning CPU/GIL/SQLite I/O. On a fast box it's invisible; on a low-end redistributed box it adds up.

---

3. Design — the Adaptive Inference Governor

A single policy object that every model call routes its *decision* through. It does **not** replace `reasoning_modes`, `model_tier`, or `hardware_profile`; it consumes them.

3.1 Inputs (all already available at runtime)

- **Model capability** — from the model itself: GGUF metadata `tokenizer.chat_template` (already read for format detection) sniffed for `<think>`/`thinking`; else a one-shot capability probe at load. Cached per model SHA/path. Replaces the Gap A allowlist.
- **Throughput** — `model_tier.measured_tok_s()` (live EMA) + a cold-start default tier estimate from `detect_tier()` until the first real measurement lands.
- **Context** — `n_ctx` from the runtime snapshot (and `n_ctx_train` from model metadata, for Gap E).
- **Task class** — see 3.2.

3.2 Task-class taxonomy (decides whether thinking can add value)

A small classifier (deterministic, keyed on the action + call-purpose already known at the call site), three classes: 1. **STRUCTURED** — JSON/routing/plan-graph. Never thinks (already true via `structured=True`). 2. **GROUNDED_SYNTHESIS** — evidence already gathered; the call only phrases/transforms it (the `EXPLAIN_*` control actions, RAG answer synthesis, fix-patch rendering). Thinking adds nothing; force no-think regardless of speed. 3. **OPEN_REASONING** — genuine multi-step reasoning (CoT/ToT/constitutional/self-consistency, open chat on a hard question). Thinking is allowed but **budget-bounded by throughput**.

3.3 The decision (pure function)

```
policy(task_class, model_caps, tok_s, n_ctx, prompt_tokens) -> {
    think: bool,
```

```
    think_budget: int,         # max tokens allowed inside <think>
    answer_budget: int,        # reserved tokens AFTER </think>
    passes: int,               # via existing speed_passes()
}
```

Rules (all derived; thresholds reuse `ELI_SLOW_TPS/ELI_FAST_TPS` = 5/15): - `model_caps.thinks == False` → `think=False` always (no prefill needed). - `task_class == STRUCTURED or GROUNDED_SYNTHESIS` → `think=False`. - `task_class == OPEN_REASONING`: - `tok_s >= fast` → think on, full budget (today's behaviour). - `slow < tok_s < fast` → think on, `think_budget` scaled so projected wall-time ≤ target. - `tok_s <= slow` → think off (or a hard-capped `think_budget` with a guaranteed `answer_budget` reserve) so the call cannot spend its whole budget thinking. - **Always reserve `answer_budget`** so a thinking call physically cannot exhaust its budget before producing an answer (directly kills the Gap D failure mode).

3.4 Empty-think self-correction (Gap D)

In the clean path, detect "`<think>` opened, `</think>` absent, finish_reason == length" and, instead of returning empty, **retry once with `think=False`** (closed-think prefill) reusing the already-built prompt. One bounded retry, model-agnostic, replaces today's silent-empty-then-full-retry cascade.

3.5 Integration points (exact, minimal)

- `gguf_inference._no_think_prefill` — consult the governor (think flag + budgets) instead of just `structured/<1024/env`. Keep `force_no_think()` as the primitive the governor drives; keep `ELI_MODEL_THINK` as a hard user override on top.
- `gguf_inference` capability detection — add `_model_thinks()` reading chat-template metadata / probe; `_is_thinking_model` becomes a thin wrapper (metadata first, name list as last-resort fallback).
- `reasoning_modes.build_*` (the budget at 322–352) — split `target_tokens` into `think_budget` + `answer_budget` from the governor; today's single `max_tokens` becomes `answer_budget` when `think=False`.
- `engine` synthesis sites — `_compact_grounded_synthesis` and `_synthesize_answer` tag their calls `GROUNDED_SYNTHESIS`; `_run_chain_of_thought`/ToT/self-consistency tag `OPEN_REASONING`. The per-site `force_no_think` wrap at 13070 is **removed** once the governor covers it.
- `startup_hardware_optimizer.train_ctx_for_model` — prefer the model's GGUF `n_ctx_train` metadata when present; keep the filename table as fallback (Gap E).

3.6 Config / env (redistribution knobs, all optional with safe defaults)

- `ELI_SLOW_TPS` / `ELI_FAST_TPS` (exist, 5/15).
- `ELI_TURN_LATENCY_TARGET_S` (new) — soft per-turn wall-time target the budget scaler aims for; default conservative.
- `ELI_MODEL_THINK` (exists) — hard override, unchanged.
- All derived at runtime; nothing machine-specific is persisted (consistent with `_portable_settings_for_storage`).

---

4. Cross-machine behaviour matrix (what "done" looks like)

Machine / model	tok/s	`EXPLAIN_*` (grounded)	hard chat (open reasoning)
8 GB GPU + 30B MoE (the dev box)	~2	no-think, single pass, ~seconds	bounded think, ≤ target, never empty
24 GB GPU + 7B	~40	no-think (still pointless to think)	full think, full budget

Machine / model	tok/s	`EXPLAIN_*` (grounded)	hard chat (open reasoning)
CPU-only + 7B	~3	no-think	think off / hard-capped + answer reserve
Any box + **non-thinking** model	n/a	no prefill needed; normal answer	normal answer (no think tokens wasted)
Any box + **unknown-name reasoning** model	any	detected via metadata → handled	handled (no allowlist miss)

Same binary, no per-machine edits.

---

5. Phased implementation plan

Phase 1 — Capability probe (Gap A, E). Add `_model_thinks()` (chat-template metadata → probe → name fallback); prefer `n_ctx_train` in `train_ctx_for_model`. Low risk, isolated.

Phase 2 — Governor core + grounded-synthesis (Gap B/F for the logged path). Add the policy function; wire `GROUNDED_SYNTHESIS` no-think into `_no_think_prefill`; tag the two synthesis sites + coding fix-render; remove the single `force_no_think` wrap. This alone eliminates the logged cascade for `EXPLAIN_*` and `SELF_IMPROVE`.

Phase 3 — Throughput-bounded open reasoning + answer reserve (Gap B/C/D). Split think/answer budgets in `reasoning_modes`; add the empty-think one-shot recovery in `gguf_inference`.

Phase 4 — Feed the EMA from streaming + proactive debounce (Gap B sampling, Gap G). Also record decode speed on the stream path so quick-mode chat updates the EMA; debounce `persona_updater`/KG-sync to fire on change, not every tick.

Each phase is independently shippable and testable.

---

6. Verification strategy (must stay model/hardware-agnostic)

- **Unit (pure functions):** `policy(...)` truth table across `{task_class}` × `{tok_s tiers}` × `{model thinks?}`; `_model_thinks()` on fixture GGUF metadata (thinking + non-thinking); `train_ctx_for_model` prefers metadata over filename.
- **No-GGUF engine tests:** extend the existing `tests/test_*_no_gguf*.py` pattern to assert the governor's decision per action without loading a model.
- **Behavioural (the log's failures):** assert `EXPLAIN_MEMORY_RUNTIME` / `EXPLAIN_COGNITION_RUNTIME` synthesis is non-empty on the **first** pass and runs no-think; assert no second/third full-length empty generation occurs.
- **Regression guards:** `tests/test_gguf_think_stop_collision.py`, `tests/test_reasoning_mode_contract.py`, router/multi-command/background suites must stay green (current baseline: all green).
- **Speed simulation:** inject a fake `measured_tok_s()` (slow/fast) and assert the budget + think flag flip accordingly — proves cross-machine adaptivity without slow hardware.

---

7. Risks & non-goals

- **Risk: over-suppressing thinking** on genuinely hard open-reasoning turns on mid-speed boxes. Mitigation: only GROUNDED_SYNTHESIS is unconditionally no-think; OPEN_REASONING keeps think with a budget, never a hard off above `slow`.

- **Risk: capability probe cost at load.** Mitigation: prefer metadata (free); probe only if metadata is silent; cache per model hash.
- **Non-goal:** changing model selection, the agent set, persona content, or the grounding contract. This is purely the *inference policy* layer.
- **Non-goal:** removing `ELI_MODEL_THINK` — it stays as the explicit user override.

---

8. Appendix — exact references (commit a641471)

- Think detection: `eli/cognition/gguf_inference.py:219` `_is_thinking_model` (`"qwen3","deepseek-r1","r1-distill","qwq","-r1-"`).
- No-think prefill: `gguf_inference._no_think_prefill` (`structured, 0 < max_tokens < 1024`, `force_no_think()` scope, `ELI_MODEL_THINK`).
- `force_no_think` primitive: `gguf_inference` (≈255); sole caller `engine.py:13070`.
- Speed EMA: `gguf_inference.py:1158–1164` (non-stream only) → `model_tier.record_speed` (:90) / `measured_tok_s` (:104); sole consumer `speed_passes` (:120, used `engine.py:6241, 6520`); thresholds `ELI_SLOW_TPS=5/ELI_FAST_TPS=15` (`model_tier._slow_fast_thresholds`).
- Budget: `eli/cognition/reasoning_modes.py:312–352` (`mode_max_from_ctx = n_ctx*0.20/0.30`, `scale = 0.85 + 0.55*complexity`; no tok/s).
- Compact grounded synthesis: `engine._compact_grounded_synthesis` (≈12946), call ≈13066, `max_tokens=1400`, now `force_no_think`-wrapped (13070).
- Standard synthesis: `engine._synthesize_answer` (13118), non-quick `max_tokens=1800` (≈13165).
- CoT runner: `engine._run_chain_of_thought` (6131) — two `_get_chat_response` stages, no `force_no_think` (closed-think in the log is model-emergent).
- ctx table: `startup_hardware_optimizer.train_ctx_for_model` (164–197); `qwen3 → 32768` vs logged `n_ctx_train=262144`.
- VRAM reserve: `hardware_profile.py:69` `256 + (n_ctx/1024)*24 + batch*1.5`.
- Proactive churn (deterministic, no LLM): `cognition/persona_updater.py` (`:349`, `:543`, `:590`).

Session-log evidence (2026-06-17), key lines

```
EXPLAIN_COGNITION_RUNTIME:
  compact synthesis  → '<think>...' → CLEAN empty            nonstream_call_total=273.356s
 standard synthesis → '<think>...' → CLEAN empty (x2)     nonstream_call_total=539.994s / 539.959s
 CoT scratchpad    → '<think>\n\n</think>\n\n...'       nonstream_call_total=470.896s  (4073 chars)
 CoT final       → '<think>\n\n</think>\n\nThe pipeline' nonstream_call_total=502.704s  (4071 chars → ~2 tok/s)
SELF_IMPROVE coding fix-gen:
  '<think>...' → empty   max_tokens=1536 / 4000           nonstream_call_total=231.122s / 445.702s
Load:
  llama_context: n_ctx_seq (20480) < n_ctx_train (262144)
```